

Web Application Security Testing with Burp Suite and OWASP ZAP

Tools Used

- Burp Suite: A web application security testing tool.
- OWASP ZAP: An open-source web application security scanner.

SITES TESTED

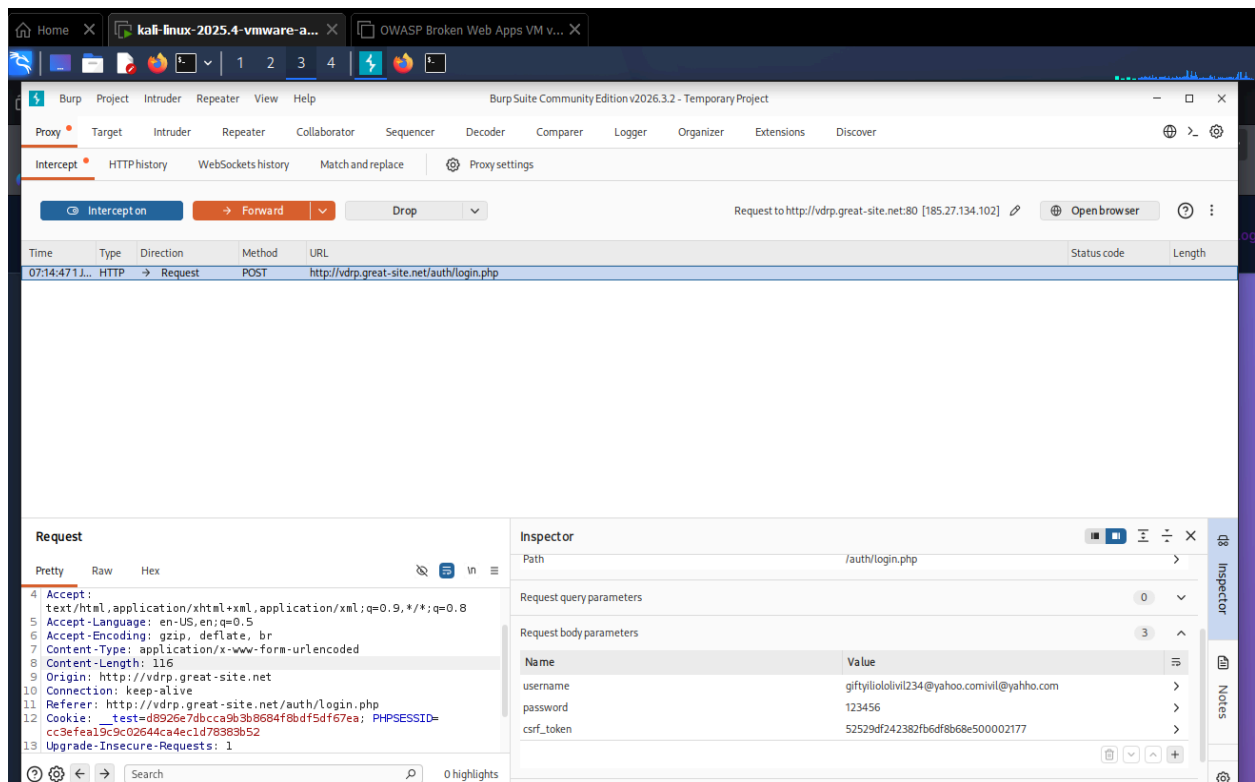
great-sit.org

Owasp broken web Application(Metilda)

Observing how Burp Suite captures and displays the traffic.

Exercise 1:

- Document the HTTP request and response headers for the home page of the target application. What information do you find in these headers?



HTTP request and response headers

POST /auth/login.php HTTP/1.1

Host: vdrp.great-site.net

User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101
Firefox/140.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate, br

Content-Type: application/x-www-form-urlencoded

Content-Length: 116

Origin: http://vdrp.great-site.net

Connection: keep-alive

Referer: http://vdrp.great-site.net/auth/login.php

Cookie: test=d8926e7dbcca9b3b8684f8bdf5df67ea;
PHPSESSID=cc3efea19c9c02644ca4ec1d78383b52

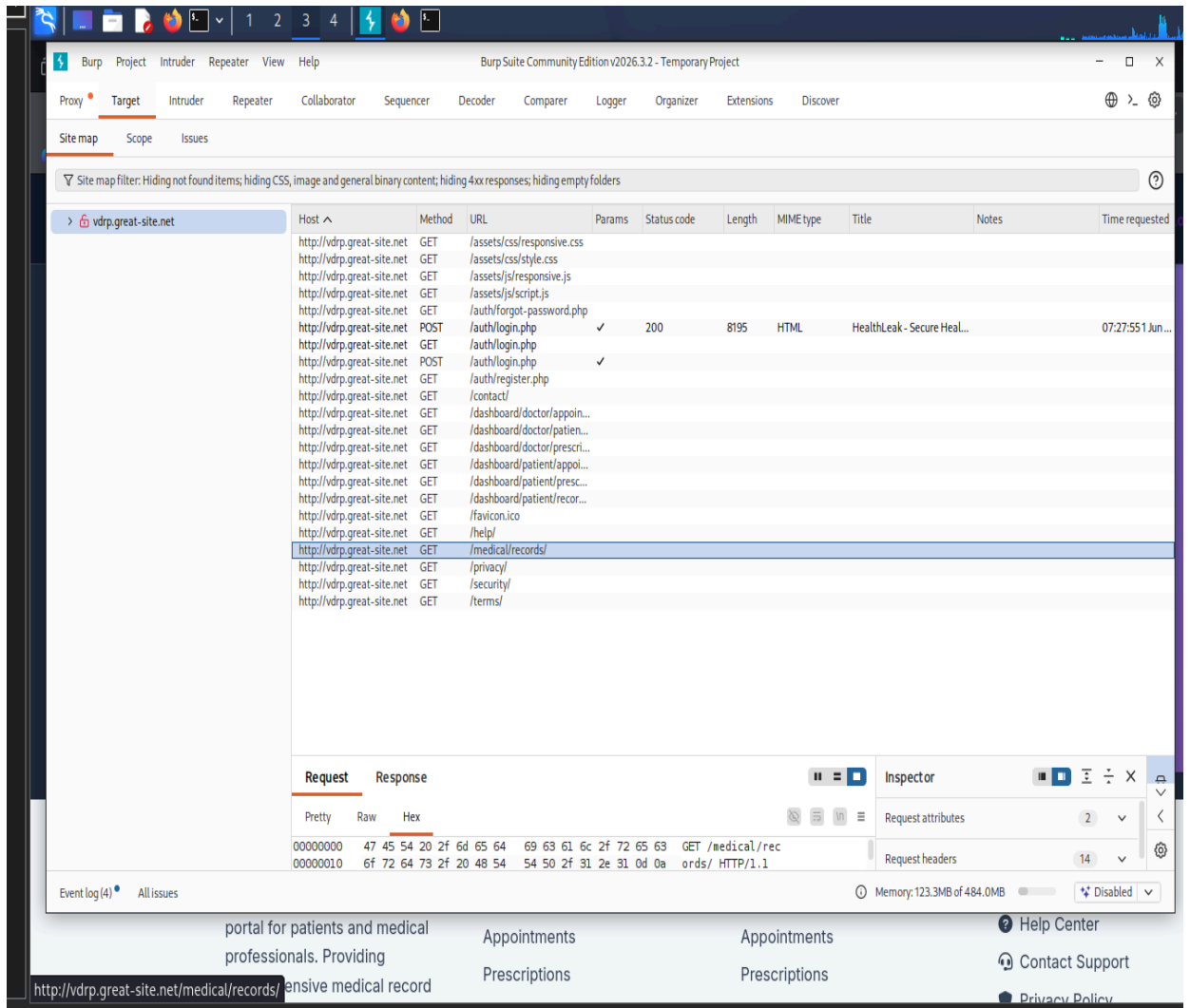
Upgrade-Insecure-Requests: 1

Priority: u=0, i

username=giftyiliolivil234%40yahoo.comivil%40yahho.com&password=1
23456&csrf_token=52529df242382fb6df8b68e500002177

Exercise 2:

- List the URLs discovered during the spidering process. Did you find any hidden or interesting pages?



Intresting URLs discovered:

<http://vdrp.great-site.net/auth/login.php>

<http://vdrp.great-site.net/auth/register.php>

<http://vdrp.great-site.net/auth/forgot-password.php>

<http://vdrp.great-site.net/dashboard/doctor/appointments/>

<http://vdrp.great-site.net/medical/records/>

<http://vdrp.great-site.net/dashboard/doctor/prescriptions/>

<http://vdrp.great-site.net/dashboard/patient/records/>

<http://vdrp.great-site.net/contact/>

2. Active Scanning:

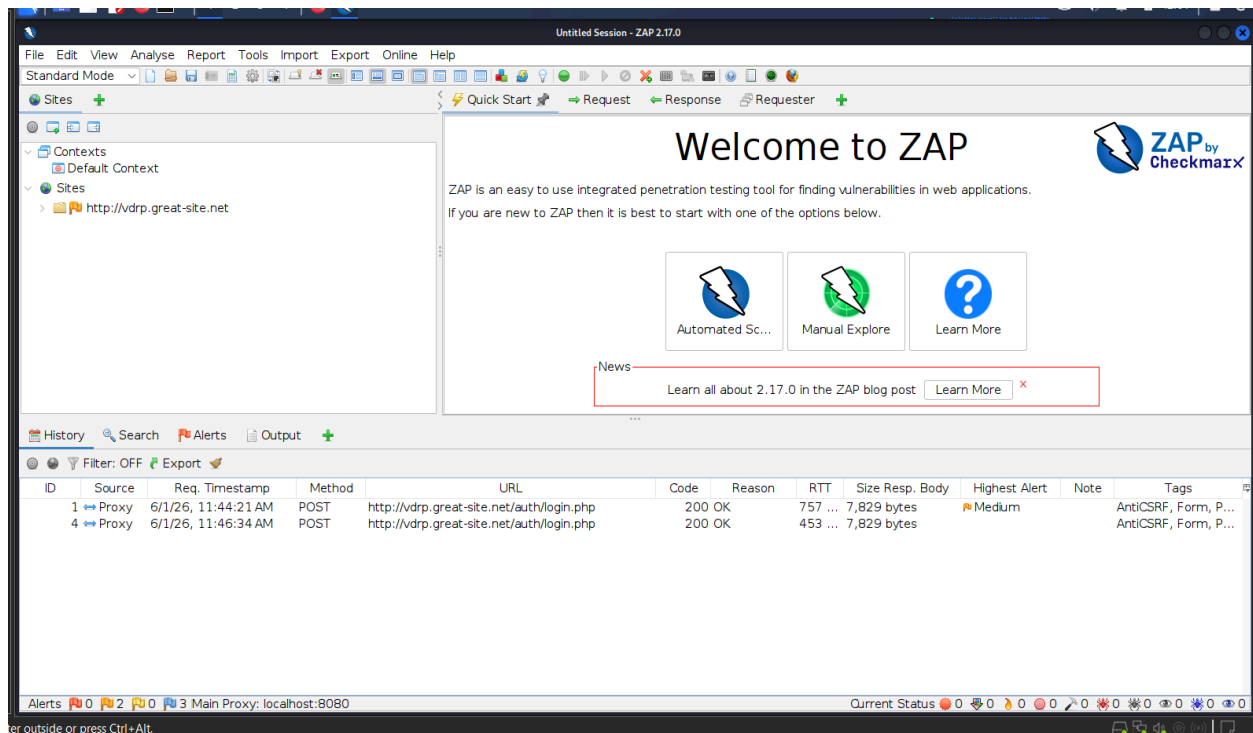
- After spidering, select the site and choose "Scan" to start an active scan.
- Review the scan results to identify any vulnerabilities found.

Exercise 3:

- What vulnerabilities were detected by Burp Suite? Choose one vulnerability and explain how it could be exploited.

Exercise 4:

- Capture and analyze the traffic with OWASP ZAP. What differences do you notice compared to Burp Suite?



1. OWASP ZAP gives automatic alerts faster displaying alerts and discovered URLs.
 2. OWASP ZAP is easier to use but not that detailed like Burp suite.
 3. OWASP ZAP is not displaying the HTTP request/response headers directly on the screen, unlike Burp Suite where headers are immediately visible in the Intercept, HTTP History, or Repeater tabs.
 4. Burp SUITE is more manual but well detailed although a bit overwhelming for a beginner on how to operate.
-

Step 4: Using OWASP ZAP for Vulnerability Scanning

1. Automated Scanner:
 - Utilize the “Quick Start” feature to run an automated scan on the target web application.
 - Monitor the alerts generated by ZAP during the scan.

Exercise 5:

- Review the vulnerabilities identified by OWASP ZAP. Which tools detected the same vulnerabilities? What are the potential impacts of these vulnerabilities?

Screen shots of Automatic scan

The screenshot shows the ZAP 2.17.0 interface during an automated scan. The 'Automated Scan' window is open, displaying the URL to attack: `http://vdrp.great-site.net/auth/login.php`. The scan progress bar is at 100%. Below the progress bar, a table lists the processed URLs and their methods.

Processed	Method	URI	Flags
●	GET	<code>http://vdrp.great-site.net/auth/login.php</code>	Seed
●	GET	<code>http://vdrp.great-site.net/robots.txt</code>	Seed
●	GET	<code>http://vdrp.great-site.net/sitemap.xml</code>	Seed
●	GET	<code>http://vdrp.great-site.net/aes.js</code>	
●	GET	<code>http://vdrp.great-site.net/</code>	
●	GET	<code>http://vdrp.great-site.net/medical/records/</code>	
●	GET	<code>http://vdrp.great-site.net/dashboard/patient/appointments/</code>	
●	GET	<code>http://vdrp.great-site.net/dashboard/patient/prescriptions/</code>	
●	GET	<code>http://vdrp.great-site.net/medical/lab/</code>	
●	GET	<code>http://vdrp.great-site.net/medical/imaging/</code>	
●	GET	<code>https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all...</code>	Out of Scope
●	GET	<code>https://fonts.googleapis.com/css2?display=swap&family=Inter:wg...</code>	Out of Scope

The status bar at the bottom shows 'Alerts: 0', '5' (yellow), '6' (blue), and 'Main Proxy: localhost:8080'.

Alert generated.

The screenshot shows the ZAP 2.17.0 interface with an alert generated. The 'Alerts' tab is selected, showing a list of alerts. The selected alert is 'User Controllable HTML Element Attribute (Potential XSS)'.

Alerts (10)

- Content Security Policy (CSP) Header Not Set (Systemic)
- Missing Anti-clickjacking Header (Systemic)
- Sub Resource Integrity Attribute Missing (Systemic)
- Cookie No HttpOnly Flag (2)
- Cookie without SameSite Attribute (2)
- X-Content-Type-Options Header Missing (Systemic)
- Authentication Request Identified
- Modern Web Application (Systemic)
- Session Management Response Identified (2)
- User Controllable HTML Element Attribute (Potential XSS)**

Parameter: `csrf_token`
Attack:
Evidence:
CWE ID: 20
WASC ID: 20
Source: Passive (10031 - User Controllable HTML Element Attribute (Potential XSS))
Input Vector:
Description:
This check looks at user-supplied input in query string parameters and POST data to identify where certain HTML attribute values might be controlled. This provides hot-spot detection for XSS (cross-site scripting) that will require further review by a security analyst to determine exploitability.
Other Info:
User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL:

The status bar at the bottom shows 'Alerts: 0', '3' (yellow), '3' (blue), '4' (blue), and 'Main Proxy: localhost:8080'.

OWASP ZAP discovered 10 Alerts which includes:

Missing Anti-clickjacking Header, Missing Content Security Policy, Missing Subresource Integrity (SRI), Cookie without SameSite Attribute, Cookie without HttpOnly Flag, Potential XSS, etc.

These vulnerabilities could lead to clickjacking, XSS attacks, session theft, and reduced browser-side security protections.

Burp Suite discovered more URLs during spidering, including login, registration, dashboard, medical record pages.

OWASP ZAP identified some of the same pages but provided fewer results.

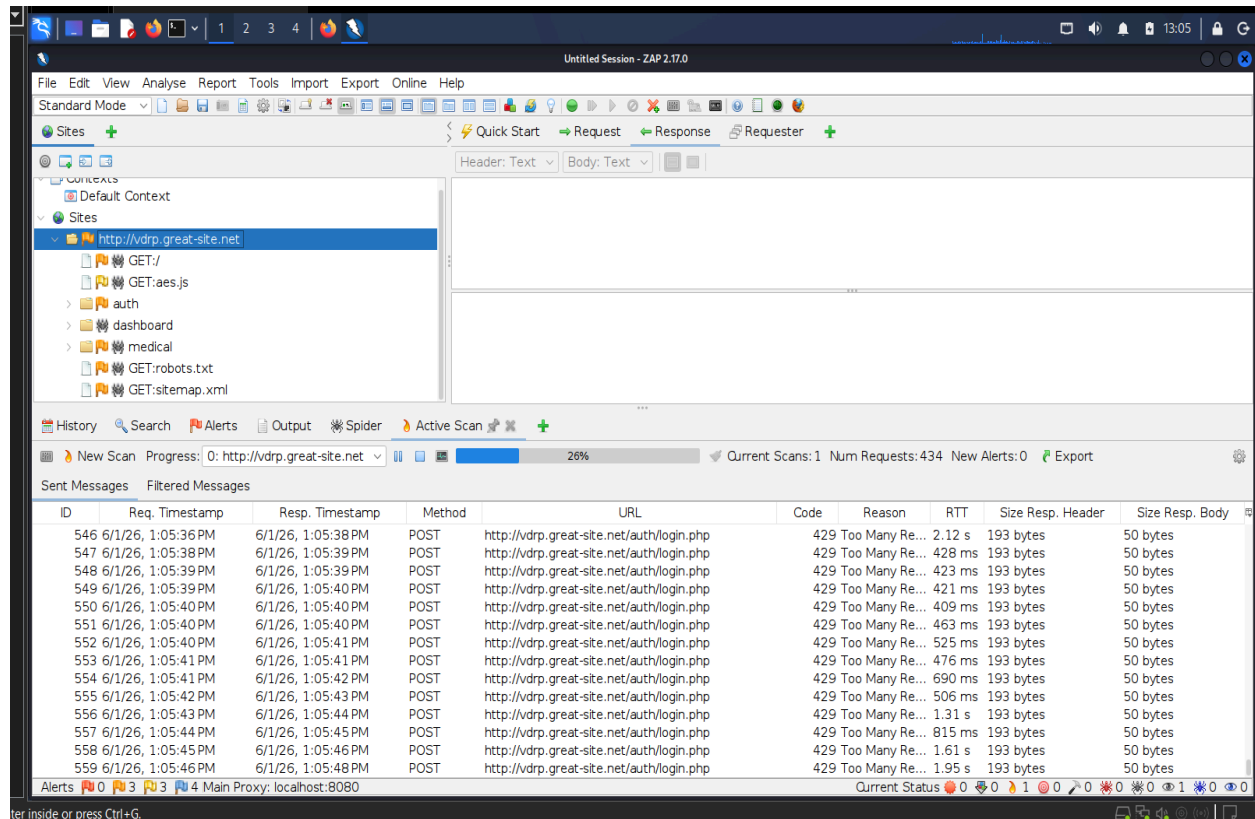
The potential impact is that exposed pages may reveal sensitive functionality that attackers could target.

2. Active Scan:

- Perform an active scan by selecting the target site and initiating the scan.
- Review the detailed reports generated.

Exercise 6:

- Compare the findings of OWASP ZAP with Burp Suite. Which tool provided more detailed information? Which tool do you prefer for vulnerability scanning? Why?



Burp Suite provided more detailed information

I prefer Burp Suite for vulnerability scanning

Why?

- OWASP ZAP is prone to freeze at times at the middle of scan and you have to restart again to continue with the task at hand.
- OWASP ZAP runs too fast and is a bit difficult to control.
- Burp Suite doesn't freeze during scan.
- Burp Suite offers deeper analysis and gives greater control to the user.

Step 5: Manual Testing Techniques

1. Fuzzing:

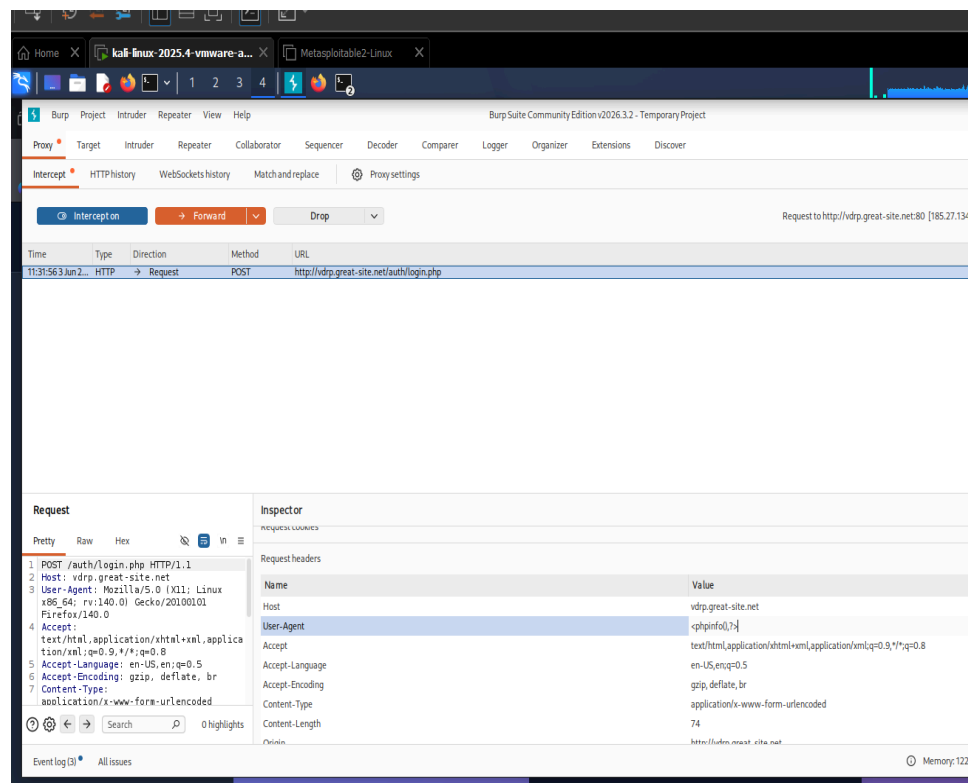
- Use both tools to perform fuzzing against input fields in the web application (e.g., login forms, search fields).
- Attempt to inject various payloads to test for common vulnerabilities like SQL injection or XSS.

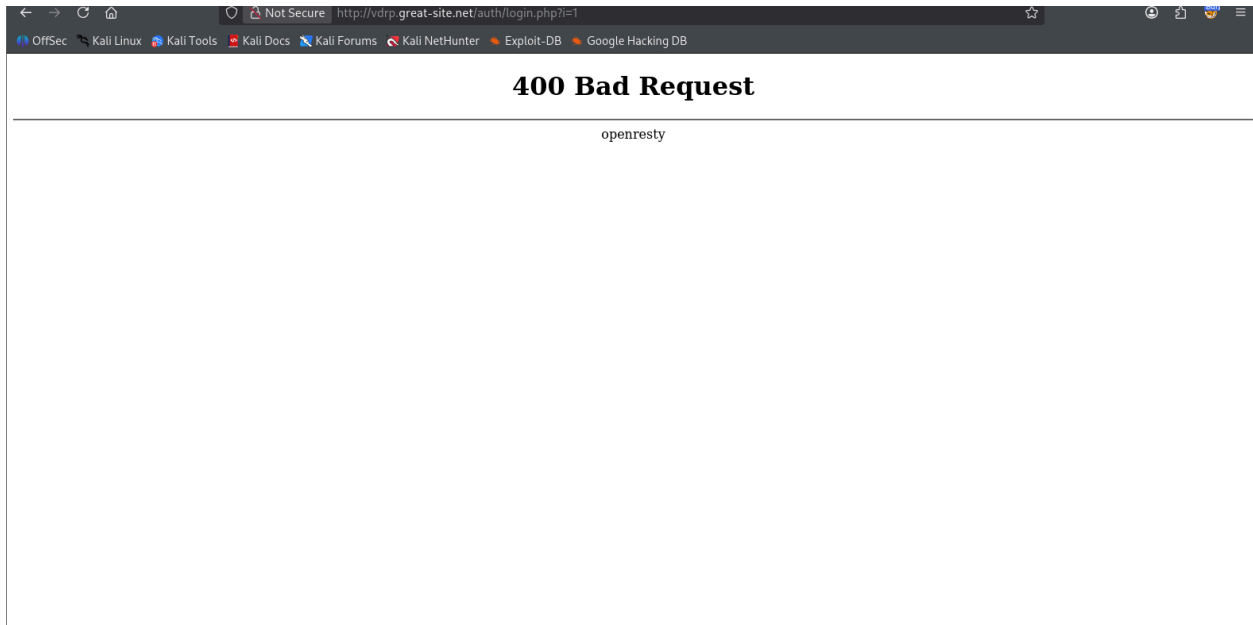
Exercise 7:

- Document any successful injections or errors encountered during fuzzing. What techniques were effective?
-

Report:

OWASP ZAP identified a Potential XSS issue involving the csrf_token parameter. However, the alert was informational with low confidence, so no successful XSS exploitation was confirmed during testing.





BURP SUIT injection was an unsuccessful response to a bad request.

Review and Documentation:

- Create a report summarizing your findings from both tools, including:
 - Identified vulnerabilities.
 - Suggested mitigations.
 - Screenshots of significant findings.

Exercise 8:

- Prepare a report detailing the vulnerabilities discovered, your methodology, and recommendations for securing the application.

LAB REPORT: WEB APPLICATION SECURITY TESTING

Methodology

The assessment was conducted using Burp Suite and OWASP ZAP. Traffic was intercepted through a proxy, the application was crawled to discover accessible pages, and automated scans were performed to identify security weaknesses.

Findings

Burp Suite Discoveries:

- /auth/login.php
- /auth/register.php
- /dashboard/doctor/appointments/
- /dashboard/doctor/prescriptions/
- /dashboard/patient/records/
- /medical/records/
- /contact/

OWASP ZAP Vulnerabilities:

- Missing Anti-clickjacking Header
- Missing Content Security Policy (CSP) Header
- Missing Subresource Integrity (SRI) Attribute
- Cookie without HttpOnly Flag
- Cookie without SameSite Attribute
- Potential XSS (Informational)
- Authentication Request Identified (Informational)

Recommendations

- Implement missing security headers.
- Configure cookies with HttpOnly and SameSite attributes.
- Validate and sanitize user input.
- Restrict access to sensitive pages and records.

- Perform regular security testing and updates.

Conclusion

The assessment revealed several accessible application pages and multiple security misconfigurations. Addressing these issues will help reduce the risk of unauthorized access and web-based attacks.